UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Quantify uncertainty in ML

Lecture 18

Course of:

Statistics and data analysis

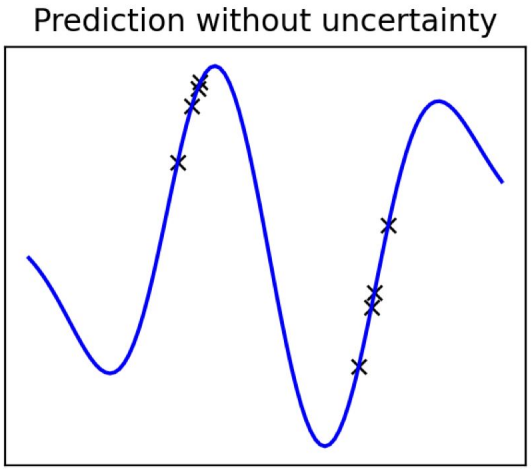
Federico De Guio - Martino Borsato

Uncertainty in ML

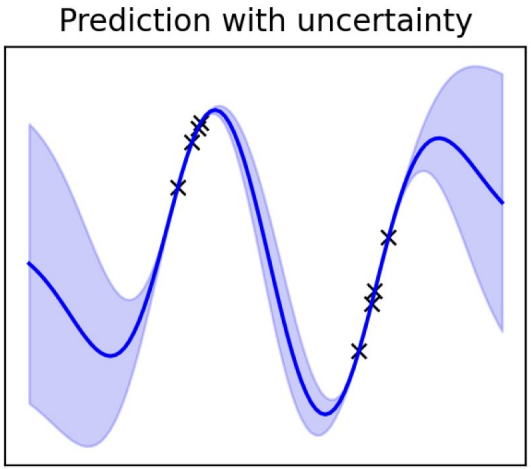
- Gal, PhD thesis
- Gal, Dropout as a Bayesian approximation
- Prince, Bayesian Neural Networks

The idea

- The ML model should **output a prediction and the corresponding uncertainty**
- The uncertainty indicates the probable interval within which an actual evaluation may be
  - (e.g. actual measurement or simulation)



e.g. neural networks

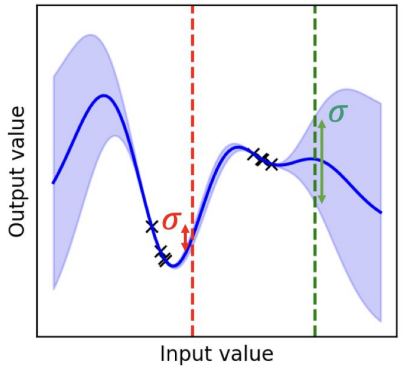


e.g. Gaussian processes

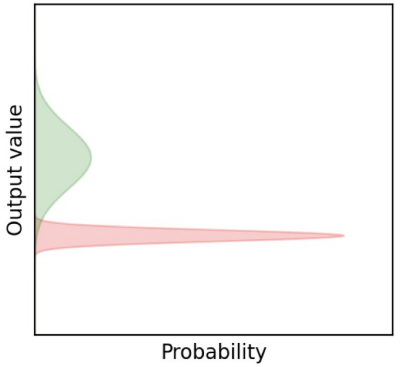
Knowing **uncertainty bands** is crucial in certain fields

- Autonomous driving
- Interpretation of medical images
- Financial forecasting
- …

Standard deviation
(Single scalar)

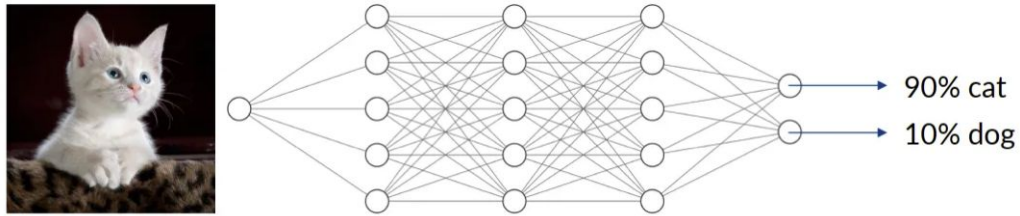


Probability distribution
(Full function)



- The probability distribution is a much more complete information, **but few ML method provide it**
- **Uncertainty quantification** (UQ) means deploy DL systems that do not only make predictions but also **provide measures of confidence in their predictions**
- ⇒ **Return a distribution instead of a prediction point**

Open set recognition

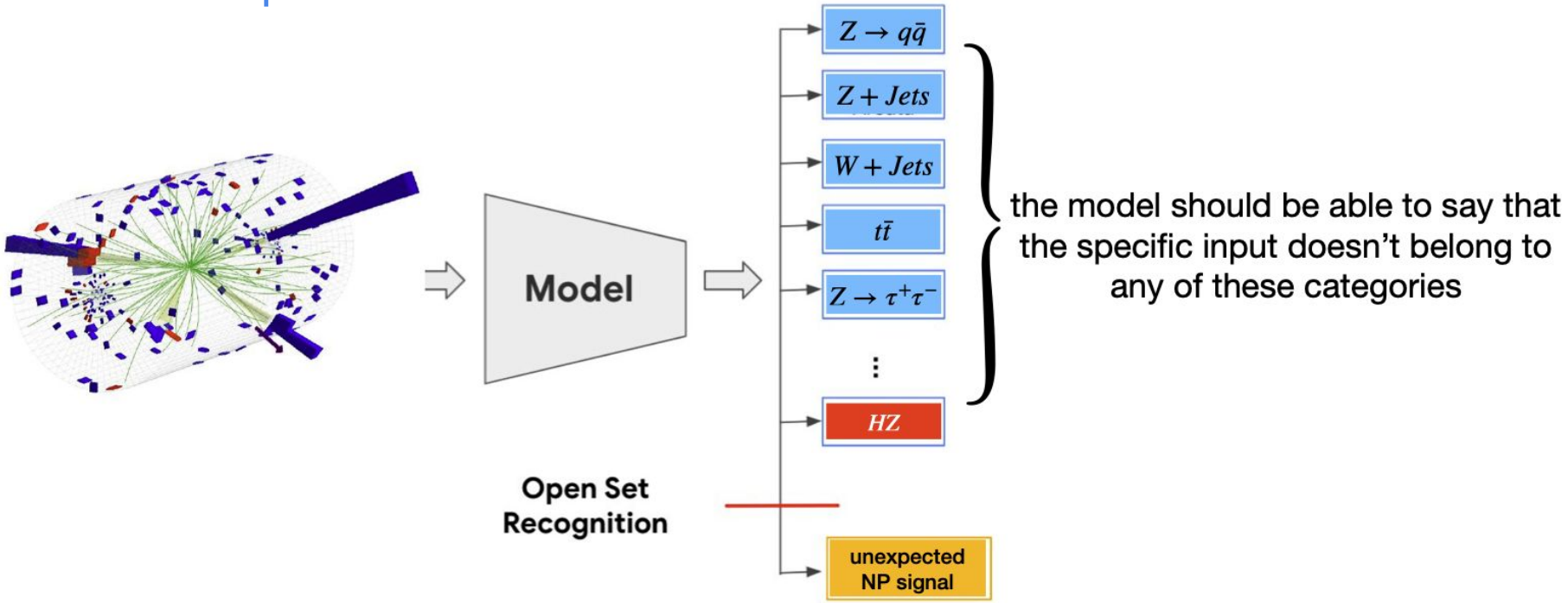


- Imagine we train a CNN to classify images of cats and dogs (see example in CNN lecture)

- What happens if we feed the model **with out of distribution samples** like a horse?
- Typically a network with a softmax output will still classify the sample to belong to one of the two classes with high score

- Model uncertainty can help deciding **how much the output can be trusted**
- **OSR** is a ML approach able to reject unknown inputs

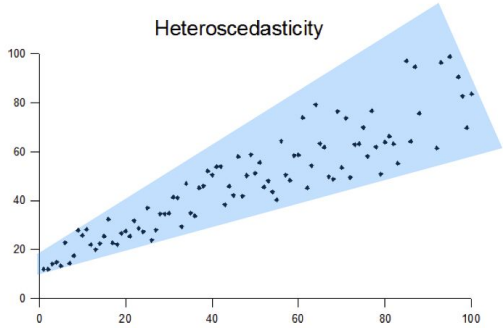
Another example



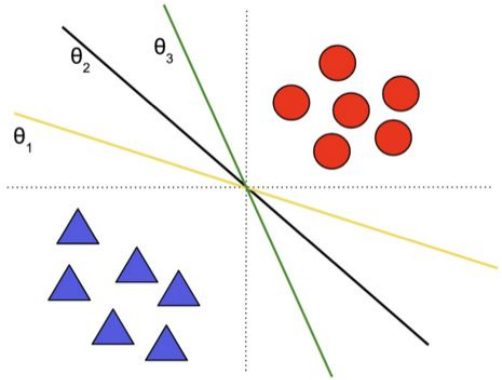
- Many sources of out of distribution shifts in data
  - Temporal changes, sampling biases, sensor/detector changes, rare events, outliers/anomalies, data corruption, differences between training dataset and real dataset…

Aleatoric and **epistemic** uncertainty

- **Aleatoric uncertainty:**
  - Intrinsic noise in the observations (ex. Sensor noise, motion noise, labelling noise, …)
  - further classified in:
    - **homoscedastic**: constant uncertainty across the input space
    - **heteroscedastic**: varies with the input
  - In general **not reducible** by adding additional data to the training set, but may be reduced with additional features describing the data



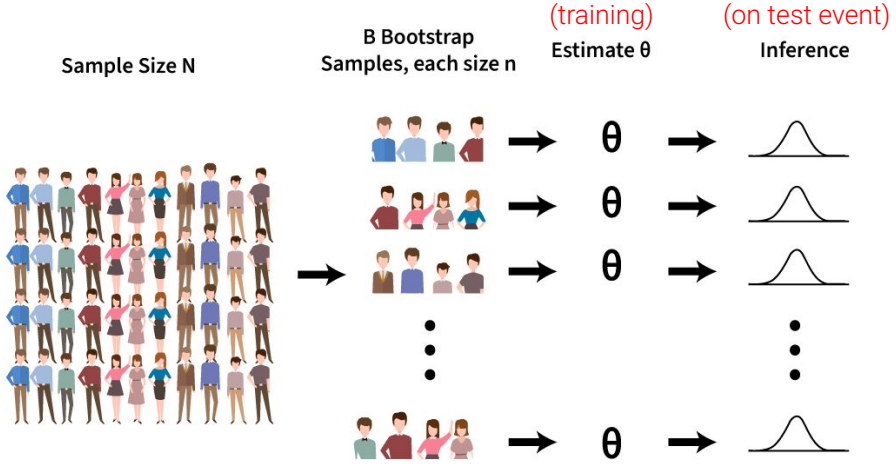
- **Epistemic uncertainty:**
  - Accounts for the **uncertainty in the model parameters**, it arises when the model is not suitably trained due to the lack of training data
  - In general it is reducible as it vanishes in the limit of infinite data (assuming model identifiability)



Ensemble methods easiest approach: **Bagging**

Frequentist approach

- **Resampling strategy via bootstrap**:
  - statistical technique that generates multiple "pseudo-datasets" by sampling with replacement from the original data (N samples out of N observations)
- **Parallel training:**
  - M identical networks (same architecture) are trained on each one of the bootstrap samples
  - **I expect to obtain M different sets of weights**
- **Inference:**
  - Run the M networks on a new event x
  - M different predictions are obtained



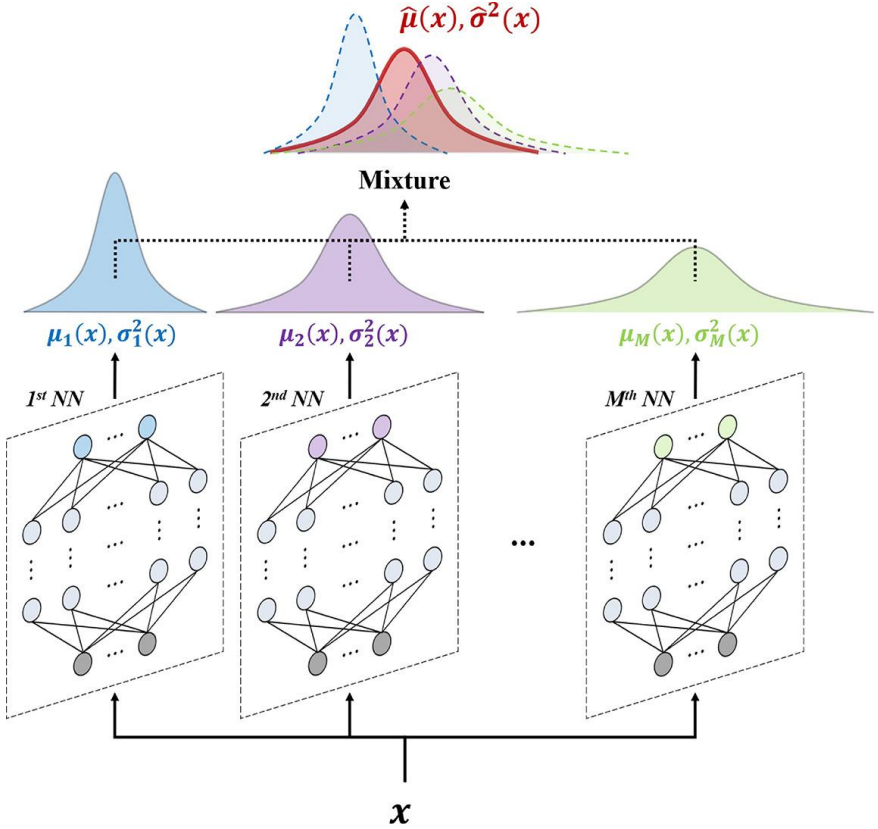
- **The final prediction** will be the mean of the M inferences, the epistemic uncertainty is the variance of the predictions

- In short: bagging allows us to **<u>simulate the variability of the entire population</u> using only our available sample**, effectively 'measuring' the uncertainty of our model without needing new experimental data

Deep ensembles

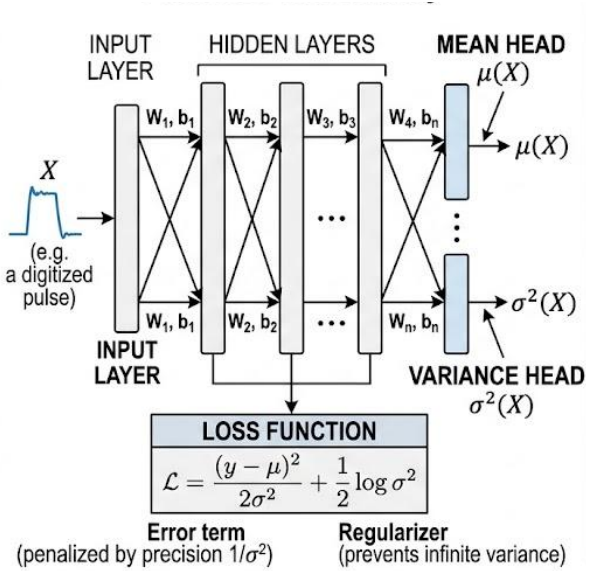
Frequentist approach

- Multiple insatances of the base architecture with different weights initialization
  - → **train on the same dataset**
- Each model output can be considered a **sample from the output distribution which expresses model uncertainty**
- Similarly to bagging, compute the mean and the sigma
- **In general ensemble methods are simple to implement in practice and readily parallelizable**
  - Don't come for free computational wise



Pointwise regression for uncertainty estimation

- Unlike standard regression, the network features a dual-head output to predict both the mean (μ) and the variance (σ^2) **for each data point**
  - **Models the noise** inherent in the input signal (**Aleatoric Uncertainty**)
- The model is trained by **minimizing the Negative Log-Likelihood**
  - This loss function penalizes the network if the predicted variance does not adequately account for the residual error, **forcing the model to "learn" where the data is noisier**
- While it provides a measure of uncertainty, **the model remains deterministic**
  - It captures the noise in the data but fails to account for Epistemic Uncertainty (model ignorance), as it cannot signal when it encounters out-of-distribution samples
- A more flexible version of pointwise regression is **Quantile Regression**
  - no need to place any assumptions on the shape of the distribution
  - Adaptive approach



Towards a Bayesian approach to uncertainty

Motivation for Bayesian methods

- In the **maximum likelihood approach**: establish the best model parameters by finding the minimum of the loss function using gradient descent.
- **Potential disadvantages** of this scheme are:
  - **No account for any prior** information we have about the parameter values
  - The final choice of the parameters is somewhat arbitrary → many functions fit well, but extrapolations diverge
  - **No estimation of uncertainty** → we would like predictions to be less certains when we are far from training points

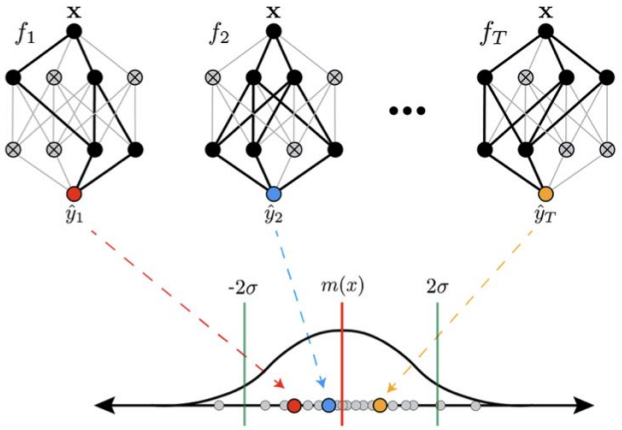
- Reject the idea of finding a single set of parameters → **acknowledge that some of them are more probable than others** depending on the degree to which they are compatible with data
  - Find **probability distributions over possible parameters in the training process**

- Need some approximation to **access the expression of the posterior** (non-tractable integrals)
  - Approximate the posterior as a collection of samples (**Sampling approach**, MCMC)
  - Approximate the posterior with a tractable form and manipulate its parameters so that it is as close as possible to the true posterior (**Variational approach**)

MonteCarlo dropout

- A very simple and fast method to sample **N independent models without requiring multiple and independent trainings: keep dropout enable at test time** and not only during training
  - Train model with dropout layers
  - Activate dropout layers at test time
  - Repeat test T times

- **Approximate Variational Inference** [Gal et al]:
  - MC Dropout is mathematically equivalent to a variational approximation approach
  - By keeping Dropout active at test time, we are effectively **sampling from the posterior distribution of the network's weights**, which is otherwise intractable
- **Epistemic Uncertainty Estimation:**
  - Each "forward pass" represents a **sample from the model space**
  - The variance across multiple stochastic passes provides a **direct measure of the model ignorance**



$$\mu(x) = \frac{1}{T}\sum_{i=1}^{T} p_i(y \mid x) \longrightarrow \text{Best estimate}$$

$$\sigma^2(x) = \frac{1}{T-1}\sum_{i=1}^{T} (p_i(y \mid x) - \mu(x))^2 \longrightarrow \text{Uncertainty}$$

Bayesian Neural Networks: a whole family of methods

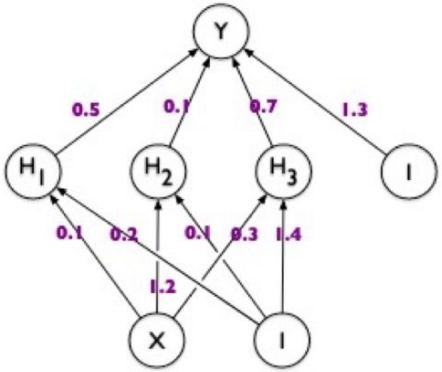
- As for the MC dropout, the prediction and uncertainty are **evaluated by averaging over repeated evaluation of the network**

- Weights are randomly drawn from a probability distribution, for each evaluation
- The probability distribution is tuned during training, according to Bayesian rules

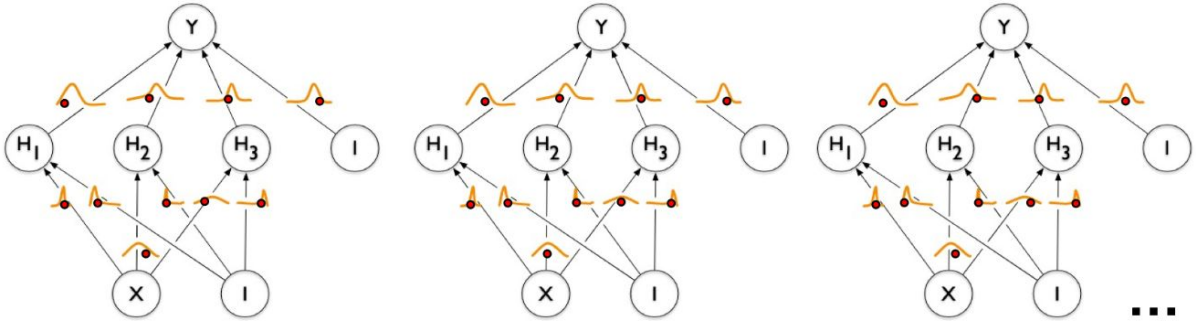
- One example: **Bayes by backprop** [Blundel et al]
  - Train Bayesian NN (BNN) using the same techniques used for the standard networks

Bayes by Backprop - **inference**

Regular neural network:
Weights are fixed.



Bayes by backprop:
Weights are drawn from **Gaussian distributions.**
The Gaussian distributions are fixed during inference, but the weights change (randomly) for each evaluation.



Each weight w_i has a different Gaussian distribution, parameterized by μ_i, ρ_i:

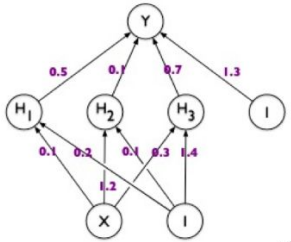
$$w_i = \mu_i + \sigma_i \epsilon_i$$

$$\epsilon_i \sim \mathcal{N}(0,1) \quad \sigma_i = \log(1 + e^{\rho_i})$$

Figures adapted from
Blundell et al., arXiv:1505.05424 (2015)

Bayes by Backprop - **training**

Regular neural network:
The weights themselves are updated.



$$w_i' = w_i - \alpha \frac{\partial \mathcal{L}}{\partial w_i}$$

Loss function:
Average error over the training data set

$$\text{MSE}=\mathcal{L} = \frac{1}{N}\sum_{j=1}^{N}(y_j - f_{\boldsymbol{w}}(\boldsymbol{x}_j))^2$$

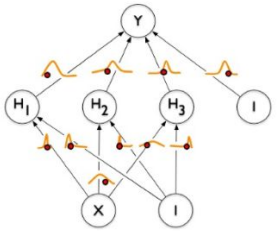
Number of examples in training set

Neural network prediction

Bayes by Backprop:
The parameters of the probability distribution (μ_i and ρ_i) are updated.

Step 1: Draw random weights

$$w_i = \mu_i + \epsilon_i \log(1 + e^{\rho_i})$$
$$\epsilon_i \sim \mathcal{N}(0,1)$$



Step 2: Update parameters

$$\mu_i' = \mu_i - \alpha\left(\frac{\partial \tilde{\mathcal{L}}}{\partial w_i} + \frac{\partial \tilde{\mathcal{L}}}{\partial \mu_i}\right)$$

$$\rho_i' = \rho_i - \alpha\left(\frac{\partial \tilde{\mathcal{L}}}{\partial w_i}\frac{\epsilon_i}{(1+e^{-\rho_i})} + \frac{\partial \tilde{\mathcal{L}}}{\partial \rho_i}\right)$$

SGD

$$\tilde{\mathcal{L}} = \frac{1}{N}\sum_{j=1}^{N}(y_j - f_{\boldsymbol{w}}(\boldsymbol{x}_j))^2 + \frac{1}{N}\left(\sum_i \log\left(\frac{e^{\frac{(w_i-\mu_i)^2}{\sigma_i^2}}}{\sigma_i}\right) - \log(P_0(\boldsymbol{w}))\right)$$

P_0: Prior on the weights

Bayes by Backprop - **ELBO loss function**

KL loss (complexity cost)

$$\tilde{\mathcal{L}} = \frac{1}{N}\sum_{j=1}^{N}(y_j - f_{\boldsymbol{w}}(\boldsymbol{x}_j))^2 + \frac{1}{N}\left(\sum_i \log\left(\frac{e^{\frac{(w_i-\mu_i)^2}{\sigma_i^2}}}{\sigma_i}\right) - \log(P_0(\boldsymbol{w}))\right)$$

Accuracy term:

- Depends on the training data
- Makes the neural network **fit the data**

Regularization term:

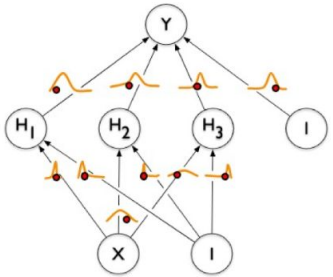
- Independent of the training data
- Tends to make the Gaussian distribution of weights **similar to the prior**
  (Typical prior: Gaussian mixture)

$$P_0(\boldsymbol{w}) \propto \Pi_i \left(\pi \frac{e^{-w_i^2/\sigma_1^2}}{\sigma_1} + (1-\pi)\frac{e^{-w_i^2/\sigma_2^2}}{\sigma_2}\right)$$

- As N increases, **the evidence provided by the data** (Likelihood) **accumulates, making the cost of misfitting the data much higher than the cost of deviating from the prior**
- This shifting balance **forces the posterior distribution to become more data-driven**

Bayes by Backprop - **Summary**

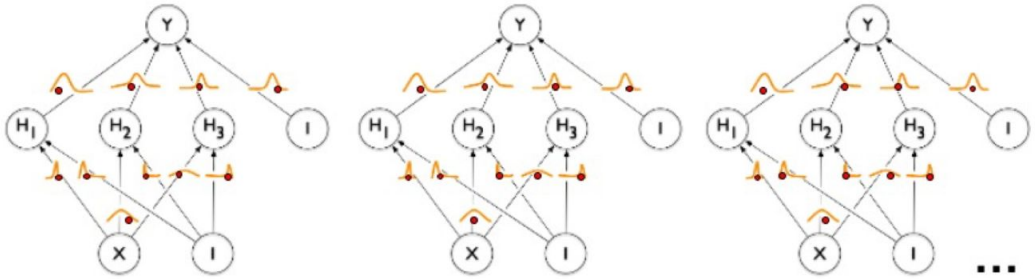
Training:
Tune the Gaussian probability distribution of the weights



$$\mu_i' = \mu_i - \alpha \left(\frac{\partial \tilde{\mathcal{L}}}{\partial w_i} + \frac{\partial \tilde{\mathcal{L}}}{\partial \mu_i} \right)$$

$$\rho_i' = \rho_i - \alpha \left(\frac{\partial \tilde{\mathcal{L}}}{\partial w_i} \frac{\epsilon_i}{(1 + e^{-\rho_i})} + \frac{\partial \tilde{\mathcal{L}}}{\partial \rho_i} \right)$$

Inference:
Draw random weights for each evaluation
Use **mean** and **standard deviation** to evaluate **prediction** and **uncertainty**



$$w_i = \mu_i + \epsilon_i \log(1 + e^{\rho_i})$$

$$\epsilon_i \sim \mathcal{N}(0, 1)$$

- In general
  - Requires **2x more parameters** with respect to a regular NN (μ_i, ϱ_i instead of w_i)
  - **Training is more difficult**: much more sensitive to hyperparameters, such as the prior

What about the VAE?

- BNN and VAE use the **same "mathematical engine"**
  - **VAE** uses the stochastic approach in the bottleneck, the rest of the network is rigid
    - → **Uncertainty in the representation** (latent space z)
      → focus on data generation and anomaly detection
  - **Bayes By Bp** transform the whole network in a stochastic object (every connection sees a distribution)
    - → **Uncertainty in the model** (weights) → focus on robust inference and epistemic uncertainty
- Both leverage the **Reparameterization Trick** and **ELBO** optimization to transform intractable Bayesian problems into scalable gradient-based learning

Variational Inference: VAE vs. Bayes by Backprop

Feature	Variational Autoencoder (VAE)	Bayes by Backprop (BNN)
Stochastic Element	**Latent Space (z)**	**Model Parameters (θ)**
Primary Goal	Learning a compressed **data distribution**.	Learning a distribution over **model parameters**.
Uncertainty Type	Focus on **data representation** and anomalies.	Focus on **model ignorance** (epistemic).
Mathematical Trick	**Reparameterization Trick** applied to z.	**Reparameterization Trick** applied to θ.
Optimization	**ELBO** (Rec. Loss + KL on z).	**ELBO** (NLL + KL on weights θ).
Output	A reconstructed sample from latent space.	A prediction from a stochastic ensemble of weights.

How to obtain an **uncertainty band from the VAE**

- **Can VAE be used to estimate an uncertainty band on the output?**
  - For each input pulse, the VAE encoder provides mu and sigma that define a gaussian
  - Randomly sample points from that gaussian and propagate through the decoder
  - The collection of output pulses defines the uncertainty band

- **Can the decoder be used as a good pulse generator? How to check the generator quality?**
  - Sample the dimensions in the latent space to generate a full collection of pulses
  - Measure the distribution of the pulses parameters
  - Compare them with those from the input dataset

To recap

- We started with VAEs to **model data uncertainty** (latent space)
- Moved to MC Dropout to **approximate model uncertainty**
- Arrived at Bayes by Backprop, which rigorously applies the **same VAE formalism directly to the network's weights**
- Aware BNN library exist both in pytorch (pyro) and tensorflow (tensorflow_probability)

Examples…

Bayes by backprop - implementation by hand

- Full regression example here

